

Project Presentation 2

H-Neurons in Small-Scale Models

Luis Felipe da Silva Carlos Pereira – RA 272919

Charles Cavalcante Alcarde – RA 800181

Pedro Henrique Guerra – RA 298988

Wed 20th May, 2026



Motivation and Original Work

Gao et al. (2025) arXiv:2512.01797

FEEC/UNICAMP

- ▶ Large Language Models can generate fluent but factually incorrect text with apparent confidence, a phenomenon known as **hallucination**.
- ▶ Gao et al. (2025) showed that an extremely sparse subset of FFN neurons, less than 0.1% of the total, can predict whether the model will hallucinate.
- ▶ These neurons were named **H-Neurons**, or *Hallucination-associated Neurons*.
- ▶ The identification is performed “from the inside out”: internal FFN activations provide a signal that precedes hallucination in the model output.
- ▶ The original study validated this phenomenon on large models ranging from 7B to 70B parameters, including Mistral, LLaMA, and Qwen.

The original paper leaves a central question unanswered:

Do H-Neurons exist in small-scale models?

Or are they an emergent phenomenon exclusive to large models?

- ▶ The smallest model evaluated by Gao et al. has 7B parameters.
- ▶ Models below this scale have not been systematically investigated.
- ▶ The Pythia family, developed by EleutherAI, provides models ranging from 70M to 6.9B parameters.
- ▶ Because Pythia models are trained on the same data and training procedure, they enable a methodologically controlled comparison across scale.

- ▶ **H1 Existence:** H-Neurons exist even in small-scale models, suggesting that the phenomenon is related to the transformer architecture itself rather than being exclusive to large models.
- ▶ **H2 Scale:** Classifier accuracy based on H-Neurons increases with model size, even within the small-scale range investigated here.
- ▶ **H3 Generalization:** H-Neurons identified on TriviaQA generalize to different domains, such as NQ-Open, suggesting that the captured signal is structural rather than dataset-specific.

These hypotheses correspond to the experimental hypotheses defined in the notebook.

We aim to investigate whether there are FFN neurons associated with hallucination in Pythia-6.9B.

Each example is represented as a triplet:

$$\mathcal{D} = \{(q_i, r_i, y_i)\}_{i=1}^{100}$$
$$y_i = \begin{cases} 0, & \text{faithful/correct answer} \\ 1, & \text{hallucinated/incorrect answer} \end{cases}$$

The central idea is:

text $\rightarrow$ internal FFN activations $\rightarrow$ CETT $\rightarrow$ L1 classifier

The classifier does not operate directly on text. Instead, it receives internal neural activations extracted from the model.

The final dataset contains 100 balanced examples:

50 correct examples + 50 hallucinated examples

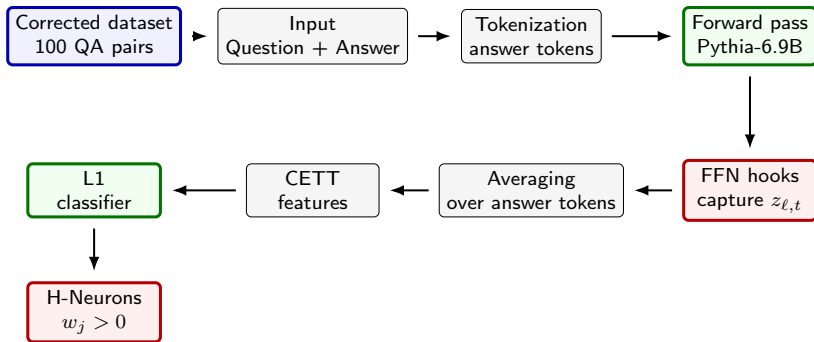
The answer r_i used as model input is selected according to the following rule:

$$r_i = \begin{cases} \text{first model-generated answer,} & y_i = 0, n_{\text{correct}} = 10 \\ \text{shortest valid textual gold answer,} & y_i = 0, n_{\text{correct}} \in \{8, 9\} \\ \text{hallucinated model-generated answer,} & y_i = 1, n_{\text{correct}} = 0 \end{cases}$$

This guarantees that hallucinated examples are also passed to the network with an explicit answer.

Overall Experimental Workflow

FEEC/UNICAMP



Pythia only processes the input sequence; learning occurs in the L1 classifier using the extracted neural activations.

For each example, the input sequence sent to the network is:

$$x_i = \text{Question: } q_i \backslash \text{n Answer: } r_i$$

The code identifies the answer tokens by comparing:

$$\text{prompt}_i = \text{Question: } q_i \backslash \text{n Answer:}$$

$$x_i = \text{prompt}_i + r_i$$

If the prompt contains p_i tokens and the full sequence contains L_i tokens, then the answer-token set is:

$$\mathcal{A}_i = \{p_i + 1, \dots, L_i\}$$

In the implementation, this corresponds to the interval:

$$[\text{answer_start}, \text{answer_end})$$

which is used to select the activations associated with answer tokens.

Pythia-6.9B follows the GPT-NeoX architecture, composed of decoder-only Transformer blocks.

$$L = 32 \quad d_{\text{model}} = 4096 \quad d_{\text{ffn}} = 16384$$

Each layer contains an FFN/MLP sublayer with the expansion:

$$4096 \longrightarrow 16384 \longrightarrow 4096$$

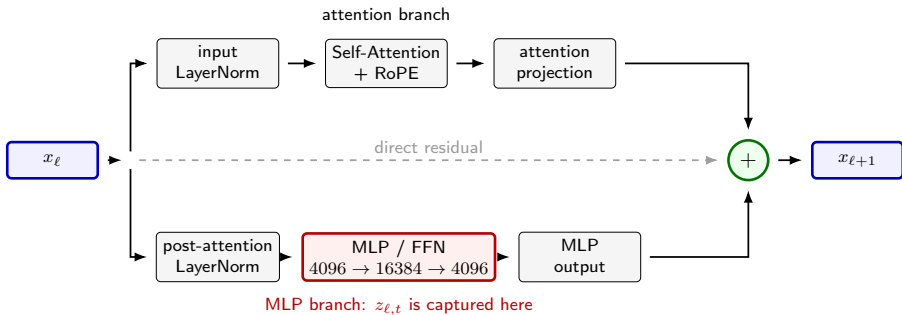
Thus, the total number of FFN neurons analyzed is:

$$N = L \times d_{\text{ffn}} = 32 \times 16384 = 524288$$

The extracted activation matrix has shape:

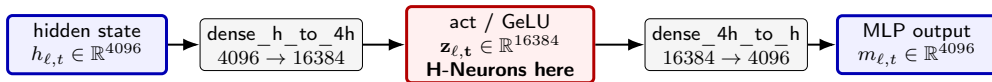
$$Z \in \mathbb{R}^{100 \times 524288}$$

GPT-NeoX/Pythia-6.9B Layer: Parallel Residual Structure



$$x_{\ell+1} = x_\ell + \text{Attention}(\text{LN}_{\text{attn}}(x_\ell)) + \text{MLP}(\text{LN}_{\text{mlp}}(x_\ell)).$$

Zoom into the MLP: Location of H-Neurons



$$u_{\ell,t} = W_{\ell}^{up} h_{\ell,t} + b_{\ell}^{up} \quad z_{\ell,t} = \text{GELU}(u_{\ell,t}) \quad m_{\ell,t} = W_{\ell}^{down} z_{\ell,t} + b_{\ell}^{down}$$

each component is an FFN neuron

The code registers a hook on `module.act`. Therefore, it captures $z_{\ell,t}$, before the `dense_4h_to_h` projection.

In the implementation, hooks are registered on:

```
module.act
```

and also on:

```
module.dense_4h_to_h
```

The main location used for H-Neuron analysis is:

```
module.act
```

because it captures:

$$z_{\ell,t} = \text{GELU} (W_{\ell}^{up} h_{\ell,t} + b_{\ell}^{up})$$

Then, only activations corresponding to answer tokens are retained:

$$t \in \mathcal{A}_i$$

For each example i , layer ℓ , and neuron j , we compute:

$$\bar{z}_{i,\ell,j} = \frac{1}{|\mathcal{A}_i|} \sum_{t \in \mathcal{A}_i} z_{\ell,t,j}$$

If the answer contains multiple tokens, their activations are aggregated by averaging.

Each example is therefore represented as:

$$\bar{z}_i = [\bar{z}_{i,1}, \bar{z}_{i,2}, \dots, \bar{z}_{i,32}]$$

After concatenating all 32 layers:

$$\bar{z}_i \in \mathbb{R}^{524288}$$

The simplified CETT metric normalizes the activation within each layer:

$$\text{CETT}_{i,\ell,j} = \frac{|\bar{z}_{i,\ell,j}|}{\|\bar{z}_{i,\ell}\|_2 + \varepsilon}$$
$$\|\bar{z}_{i,\ell}\|_2 = \sqrt{\sum_{j=1}^{16384} \bar{z}_{i,\ell,j}^2}$$

Interpretation:

$\text{CETT}_{i,\ell,j} \approx$ relative contribution of neuron j in layer ℓ

The final feature matrix is:

$$X \in \mathbb{R}^{100 \times 524288}$$

The classifier receives X_i , not the original text.
Logistic regression computes:

$$s_i = b + X_i w$$

$$p_i = P(y_i = 1 \mid X_i) = \sigma(s_i) = \frac{1}{1 + e^{-s_i}}$$

The loss with L1 regularization is:

$$\mathcal{L} = - \sum_i [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] + \lambda \sum_{j=1}^N |w_j|$$

The L1 penalty encourages many weights w_j to become exactly zero.

Each weight w_j corresponds to one global FFN neuron.

$$w_j > 0 \Rightarrow \text{increases } P(y = 1)$$

$$w_j < 0 \Rightarrow \text{decreases } P(y = 1)$$

$$w_j = 0 \Rightarrow \text{neuron discarded by L1}$$

Since $y = 1$ denotes hallucination:

$$\mathcal{H} = \{j : w_j > 0\}$$

In the code:

```
h_neuron_indices = np.where(best_clf.coef_[0] > 0)
```

$$\%H = \frac{|\mathcal{H}|}{524288} \times 100$$

After training on TriviaQA, the classifier is evaluated on:

in-domain validation

and on:

NQ-Open, out-of-domain

In the OOD evaluation:

- ▶ the L1 classifier is not re-trained;
- ▶ the scaler fitted on the original training set is reused;
- ▶ the same pipeline is applied: input, hooks, averaging, CETT, and prediction.

The goal is to assess whether H-Neurons learned in one domain retain predictive signal in a different domain.

1. The dataset was corrected to ensure an explicit question-answer input.
2. Pythia-6.9B processes the full sequence, but only answer-token activations are analyzed.
3. Hooks capture $z_{\ell,t}$ at the GeLU output of the FFN.
4. Activations are averaged over the answer tokens.
5. CETT converts activations into layer-wise relative neuron contributions.
6. L1 logistic regression selects sparse, predictive neurons.
7. H-Neurons are defined as neurons with positive classifier weights:

$$w_j > 0$$

Central scientific question:

Do FFN neurons exhibit activation patterns predictive of hallucination?

Preliminary Results

Work in progress 2 of 7 models completed

FEEC/UNICAMP

Central finding H1 supported:

H-Neurons emerge in small-scale models. The phenomenon is not exclusive to large architectures.

Key results:

- ▶ **Pythia-410M:** 67 H-Neurons, corresponding to 0.068% of 98,304 FFN neurons. AUROC: 0.79 vs. 0.51 random baseline, a gain of +0.28.
- ▶ **Pythia-6.9B:** 123 H-Neurons, corresponding to 0.023% of 524,288 FFN neurons. AUROC: 0.76 vs. 0.54 random baseline, a gain of +0.22.
- ▶ Both models remain below the $< 0.1\%$ sparsity threshold reported in the original paper.
- ▶ The proportion of H-Neurons decreases with scale, suggesting that larger models may concentrate the hallucination signal in fewer, more specialized neurons.

H2 and H3 remain open and depend on the remaining model runs.

Thanks!